Metal-Sci: A Scientific Compute Benchmark for Evolutionary LLM Kernel Search on Apple Silicon

Anonymous Authors
Anonymous Affiliation
anonymous@example.org

Abstract

LLM-driven code search has produced impressive results on GPU ML kernels (GEMM, attention, convolution), but the optimization patterns in scientific computing are systematically different: bandwidth-bound stencils with halo tiling, register-blocked all-pairs interactions, multi-field Boltzmann updates, neighbour-list molecular dynamics with atomics, parallel-chain MCMC, and multi-kernel nonlinear PDE solvers. We present METAL-SCI, a 9-task benchmark of scientific Apple-Silicon Metal compute kernels spanning five optimization regimes. Every task ships a CPU reference, a roofline ceiling, and a naive but correct seed; the fitness signal is the geometric mean of *achieved/ceiling* across three problem sizes, with any tolerance failure forcing the score to zero. We release the harness and seeds, and report preliminary 5–25 iteration evolution runs for Claude Opus 4.7, Gemini 3-Flash, and Gemini 3.1-Pro on an Apple M1 Pro. Frontier models recover canonical optimisations on well-trodden tasks (threadgroup tiling on N-body, BGK symmetry algebra on Lattice Boltzmann, template specialisation on HMC) yet still mishandle Metal-specific concurrency syntax and stall at single-digit percent of peak on irregular-memory tasks (Lennard-Jones with cell lists). The benchmark is designed for fast iteration: each candidate compiles, runs, verifies, and scores in seconds.

1 Introduction

LLM code-search systems — FunSearch [1], AlphaEvolve [2], and recent kernel-generation work [3] — evaluate language models as *optimisers* of executable artefacts. The choice of artefact matters: KernelBench targets PyTorch ML kernels (GEMM, attention, convolution) on CUDA, where canonical implementations are heavily represented in pretraining data and the optimisation surface is dominated by tiling and warp-level reductions.

Scientific computing exposes a different surface. Stencils are bandwidth-bound and reward halo and temporal blocking. All-pairs interactions are compute-bound and reward register blocking with shared-memory cooperative loads. Lattice Boltzmann ping-pongs nine distribution functions per cell with non-trivial pull-stream indexing. Cell-list molecular dynamics touches irregular memory through atomic counters. Hamiltonian Monte Carlo runs many chains in parallel where register pressure scales with the problem dimension d. Grad-Shafranov solvers chain a max-reduction with a variable-coefficient stencil. Each pattern stresses a distinct dimension of the compiler/memory hierarchy, and canonical optimisations [5,6,7,8] differ regime-to-regime.

We target Apple Silicon's Metal compute pipeline. It is underrepresented in CUDA-centric training data — frontier models reliably mishandle Metal-specific syntax such as the `[[max_total_threads_per_threadgroup]]` kernel attribute, a real out-of-distribution generalisation test. Its unified memory model removes host-device copy plumbing, enabling sub-second compile-run-verify cycles per candidate that are essential for fast evolutionary loops. And it sup-

Preprint.

ports rich simdgroup intrinsics (`simd_max`, `simd_broadcast`) and threadgroup-memory tiling, giving the LLM a non-trivial optimisation surface.

Contributions. (i) A 9-task scientific compute benchmark over five optimisation regimes, each with a CPU reference, roofline-anchored fitness, and multi-size generalisation gate. (ii) A runtime-compiled harness that exposes compile errors, correctness diagnostics, and per-size GPU timing back to the LLM. (iii) Preliminary evolution runs across Claude Opus 4.7 and Gemini 3 on M1 Pro, qualitatively analysing which patterns each model recovers.

2 Benchmark tasks

Each task ships (a) a Metal source describing the problem, (b) a naive correct seed, (c) a CPU reference with task-specific tolerance, (d) three in-distribution sizes plus one held-out size, and (e) a roofline ceiling in either GFLOPS (compute-bound) or GB/s (bandwidth-bound). Tasks are grouped by optimisation regime.

2.1 Regular stencils (regime R1)

heat2d. Two-dimensional heat equation, 5-point stencil on $(N_x \times N_y)$ grid with Dirichlet boundaries. Per timestep, per interior cell:

$$u^{n+1}_{i,j} \;=\; u^n_{i,j} + \alpha\big(u^n_{i-1,j} + u^n_{i+1,j} + u^n_{i,j-1} + u^n_{i,j+1} - 4\,u^n_{i,j}\big)\,. \tag{1}$$

Bandwidth-bound at 8 B/cell; tests halo handling and temporal blocking.

wave3d. Three-dimensional acoustic wave equation, 7-point isotropic Laplacian, leapfrog in time:

$$u^{n+1}_{i,j,k} = 2u^n_{i,j,k} - u^{n-1}_{i,j,k} + \alpha\,\big(u^n_{i\pm1,j,k} + u^n_{i,j\pm1,k} + u^n_{i,j,k\pm1} - 6\,u^n_{i,j,k}\big), \tag{2}$$

where each $\pm$ expands to two terms. CFL stability requires $\alpha = (c\Delta t/\Delta x)^2 < 1/3$ (we use 0.18). 12 B/cell unique DRAM traffic. Tests register pressure, 2.5D blocking, and marching-axis choice.

2.2 Compute-bound (regime R2)

nbody. All-pairs gravitational N-body with leapfrog. Per body i per step:

$$\mathbf{a}_i \;=\; G\sum_j m_j\,\frac{\mathbf{r}_j - \mathbf{r}_i}{(\|\mathbf{r}_j - \mathbf{r}_i\|^2 + \varepsilon^2)^{3/2}}, \quad \mathbf{v} \mathrel{+}= \mathbf{a}\Delta t, \quad \mathbf{r} \mathrel{+}= \mathbf{v}\Delta t. \tag{3}$$

Self-interaction is masked by the softening ε. $\sim$20 FLOPs per pair; ceiling at peak FP32 GFLOPS. Tests register blocking and threadgroup cooperative loads.

hmc. Hamiltonian Monte Carlo on an anisotropic Gaussian target, $U(q) = \frac{1}{2}q^\top A q$ with $A = \Sigma^{-1}$. One thread per chain; many chains in parallel. Each HMC step draws $p \sim \mathcal{N}(0, I)$ and runs L leapfrog inner steps with stepsize ε,

$$p \leftarrow p - \tfrac{\varepsilon}{2}Aq, \quad q \leftarrow q + \varepsilon p, \quad p \leftarrow p - \varepsilon Aq, \;\cdots, \quad p \leftarrow p - \tfrac{\varepsilon}{2}Aq, \tag{4}$$

followed by a Metropolis accept/reject with $\log u_{\text{acc}} < -\Delta H$. Correctness is verified statistically (sample mean and Frobenius covariance error vs. the target). Three sizes $(d, K) \in \{(8, 16\text{K}), (16, 4\text{K}), (32, 1\text{K})\}$ probe the register-pressure boundary; at d=32 per-thread state ($\sim$512 B) competes with the register file.

2.3 Multi-field, exotic memory (regime R3)

lbm. D2Q9 Lattice Boltzmann, fused pull-stream + BGK collision, periodic BC. With velocity table $\mathbf{c}_k$ and weights w_k, per cell per step:

$$f^{\text{str}}_k(\mathbf{x}) = f^{\text{in}}_k(\mathbf{x} - \mathbf{c}_k), \quad \rho = \textstyle\sum_k f^{\text{str}}_k, \quad \mathbf{u} = \frac{1}{\rho}\sum_k \mathbf{c}_k\, f^{\text{str}}_k, \tag{5}$$

$$f^{\text{eq}}_k = w_k\rho\big(1 + 3(\mathbf{c}_k{\cdot}\mathbf{u}) + \tfrac{9}{2}(\mathbf{c}_k{\cdot}\mathbf{u})^2 - \tfrac{3}{2}\|\mathbf{u}\|^2\big)\,, \tag{6}$$

$$f^{\text{out}}_k = f^{\text{str}}_k - \tau^{-1}\big(f^{\text{str}}_k - f^{\text{eq}}_k\big)\,. \tag{7}$$

Storage is SoA: $f[k\,N_xN_y + jN_x + i]$. 72 B/cell DRAM traffic. Tests AoS/SoA layout, push vs pull streaming, and algebraic factorisations of the BGK kernel.

ising. Two-dimensional Ising model, checkerboard Metropolis Monte Carlo on a periodic $N_x \times N_y$ lattice with $\sigma \in \{\pm 1\}$ stored as int8. Per attempt at site (i, j):

$$h = \sigma_{i\pm1,j} + \sigma_{i,j\pm1} \in \{-4, \ldots, 4\}, \quad \Delta E = 2J\sigma\, h, \quad \text{accept} \iff u < \exp(-\beta\, \Delta E). \tag{8}$$

A precomputed five-entry p_{accept} table and a counter-based Murmur-fmix32 PRNG yield bit-exact CPU/GPU agreement: verification is byte-equality on the spin array. 2 B/site/sweep ceiling.

2.4 Irregular memory and atomics (regime R4)

lj. Lennard-Jones molecular dynamics with a cell-list spatial hash. Three kernels per step — `clear_cells`, `build_cells` (atomic scatter), `step` (27-neighbour-cell pair force + integration):

$$\mathbf{F}_i = \sum_{j \in \mathcal{N}(i)} -24\big(2r_{ij}^{-12} - r_{ij}^{-6}\big)\, r_{ij}^{-2}\, (\mathbf{r}_j - \mathbf{r}_i), \qquad r_{ij} < r_{\text{cut}} = 2.5, \tag{9}$$

with minimum-image periodic wrap. Cell occupancy is built via `atomic_fetch_add` on a per-cell counter. Tests load balancing, atomic contention, and the irregular access patterns of neighbour-list MD.

2.5 Multi-kernel reductions (regime R5)

gradshaf. Grad-Shafranov fixed-boundary equilibrium via Picard iteration, two kernels per outer step: an interior max-reduction $\psi_{\text{axis}} = \max_{(i,j)\in\text{int}} \psi$, then a variable-coefficient 5-point stencil with nonlinear source:

$$\psi_{\text{norm}} = \psi/\psi_{\text{axis}}, \quad J = R\, p_{\text{axis}} \cdot 4\psi_{\text{norm}}(1 - \psi_{\text{norm}}) \cdot \mathbf{1}[0 < \psi_{\text{norm}} < 1], \tag{10}$$

$$\Delta^*\psi = a_W\psi_W + a_E\psi_E + a_N\psi_N + a_S\psi_S + a_C\psi_C, \tag{11}$$

$$\psi^{n+1} = \psi^n + \omega\,(-\mu_0 RJ - \Delta^*\psi)/a_C, \tag{12}$$

with R-dependent $a_{W,E} = 1/dR^2 \pm 1/(2RdR)$, $a_{N,S} = 1/dZ^2$, $a_C = -2(1/dR^2 + 1/dZ^2)$. Tests in-kernel reduction strategies (single-TG vs simdgroup-tree) and multi-kernel composition. A bandwidth-bound smoke-test (`saxpy`) is also included to validate the harness.

3 Harness

Compile and dispatch. The harness uses PyObjC's Metal bindings and runtime-compiles `.metal` source via `MTLDevice.newLibraryWithSource`, avoiding the offline `xcrun metal` toolchain; compile errors are returned as structured strings to the LLM. Buffers are allocated in unified memory; numpy views alias buffer contents with a single copy on read-back. All dispatches for a multi-size run share one `MTLCommandBuffer`, and timings come from `GPUEndTime` − `GPUStartTime` (3 warmup, 10 timed, median reported). The chip is detected from `sysctl` and looked up in a per-family table (M1 through M4) for peak FP32 GFLOPS and DRAM bandwidth; all runs here are on Apple M1 Pro (4500 GFLOPS, 200 GB/s).

Scoring. For a candidate passing correctness at every size, $S = (\prod_i f_i)^{1/n}$ with $f_i = \text{achieved}_i/\text{ceiling}_i$. Any tolerance failure forces S=0; this hard gate prevents trading correctness for speed, and the geometric mean across sizes discourages overfitting to one regime.

Evolution loop. A single-population (μ=1+λ=1) loop. Iteration i shows the LLM the previous candidate, the incumbent best, a short history per iteration, and the original task spec. The LLM returns a Metal source block (multi-kernel tasks emit multiple `kernel` functions in one file). Candidates strictly beating the incumbent are promoted; prompts, raw responses, extended-thinking tokens, sources, and per-size JSON results are persisted. A single `call_llm` bridge dispatches Claude via the Agent SDK and Gemini via `google-genai`.

4 Preliminary results

Table 1 summarises one representative run per task on Apple M1 Pro. Scores are geometric mean of achieved/ceiling across the three in-distribution sizes; the column "best @ largest" reports the fraction-of-ceiling at the largest size of the incumbent best.

Table 1: Preliminary evolution runs on Apple M1 Pro. Score is the geometric mean of achieved/ceiling over three in-distribution sizes; correctness is a hard gate. Best @ largest is the fraction-of-ceiling at the largest in-distribution size.

Task	Model	Iters	Seed	Best	Speedup	Best @ largest
heat2d	gemini-2.5-flash	1	0.484	0.484	1.00×	—
wave3d	claude-opus-4-7	15	0.533	0.673	1.26×	77.1%
nbody	gemini-3-flash-preview	5	0.020	0.061	2.99×	17.7%
nbody	gemini-3.1-pro-preview	5	0.020	0.045	2.19×	22.0%
hmc	claude-opus-4-7	10	0.009	0.093	10.6×	21.6% (d=8)
lbm	claude-opus-4-7	25	0.395	0.576	1.46×	121.6%
lbm	claude-opus-4-6	25	0.392	0.545	1.39×	—
ising	claude-opus-4-7	10	0.066	0.075	1.13×	11.6%
lj	claude-opus-4-7	10	0.003	0.005	1.77×	1.0%
lj	gemini-3.1-pro-preview	15	0.003	0.006	1.98×	—
gradshaf	claude-opus-4-7	10	0.022	0.041	1.89×	10.7%

```
// iter 5: runtime d, D_MAX=32 layout
for (uint i=0; i<d; ++i) {
  float4 acc = Arow[0] * q4[0];
  acc = fma(Arow[1], q4[1], acc);
  /* ... 6 more, always 8 ... */
  acc = fma(Arow[7], q4[7], acc);
  f[i] = acc.x+acc.y+acc.z+acc.w;
}
// d=8: 121 GFLOPS (2.7% of peak)
```

+ template <uint D> *compile-time D*

8.0×

```
// iter 6: template <uint D>
template <uint D>
inline void run(...) {
  float q[D], p[D], f[D];
  #pragma unroll
  for (uint i=0; i<D; ++i) {
    float acc = 0.0f;
    #pragma unroll
    for (uint j=0; j<D; ++j)
      acc = fma(A[i*D+j], q[j], acc);
    f[i] = acc;
} }
if (d==8u) run<8u>(...);
else if (d==16u) run<16u>(...);
else        run<32u>(...);
// d=8: 970 GFLOPS (22% of peak)
```

Figure 1: Paradigmatic candidate evolution: HMC, Opus 4.7, iter $5 \to 6$. The structural change is one declaration — `template <uint D>` with three runtime-dispatched instantiations $d \in \{8, 16, 32\}$. Iter 5 had a manually-unrolled float4 inner loop against a fixed $D_{\max}$=32 layout (over-computing at d=8, plus a 4-way horizontal sum); iter 6 sizes `q,p,f` exactly to D so both loops fully unroll into scalar FMAs — moving d=8 from 121 to 970 GFLOPS in one iteration after five had stalled at 2–4% of peak.

The LBM 256^2 result above 100% of the published ceiling reflects working-set residency in Apple Silicon's system-level cache: the 8 B/cell roofline is DRAM-only, while the streamed 9-distribution working set fits in SLC at this size. We report it unmodified to flag the limitation of the static roofline in the multi-field, multi-step regime.

Qualitative findings. Three patterns recur across runs. *(i) Models recover canonical optimisations on well-trodden tasks.* On nbody, Gemini-3.1-Pro one-shots threadgroup-memory tiling with cooperative load and 4-way ILP unroll (Nyland 2007); on LBM, both Claude 4.6 and 4.7 independently find the BGK pairwise-symmetry algebraic factorisation; on HMC (Fig. 1), Opus 4.7 introduces a `template <uint D>` worker dispatched on runtime d at iteration 6, taking d=8 from 121 to 970 GFLOPS in a single iteration by enabling full unroll of the inner Aq matvec. *(ii) Metal-specific syntax is a real OOD failure.* On LBM, Opus 4.6 mishandles the `[[max_total_threads_per_threadgroup(N)]]` attribute placement on six independent iterations (the same compile error each time), missing a structural lever Opus 4.7 uses correctly on its first attempt. On gradshaf, an early run hit 6/10 NaN failures because the model hard-coded a 32×8 tile against a host-pinned 16×16 threadgroup; an explicit task-spec note about dispatch geometry steered the next run cleanly past the trap. *(iii) Irregular tasks remain hard.* Lennard-Jones with cell lists sits at $\sim$1% of FP32 peak after 10 iterations across both Claude and Gemini. The atomic scatter in `build_cells` and the variable-occupancy neighbour-cell loop are not patterns the models recombine effectively in our budget; this is the largest gap in the suite and an obvious target for search-strategy work.

5 Discussion

A roofline-anchored score answers "how close are we to the hardware?" in physical units; the geometric mean across sizes is hard to game by overfitting one regime. Apple Silicon's unified memory halves the host-side scaffolding compared to CUDA, enabling sub-second compile-run-verify cycles per candidate — which matters more than it sounds: an evolutionary loop with tens of candidates needs a fast *harness*, not a fast kernel. Metal's smaller, quirkier surface (simdgroup intrinsics, threadgroup attributes) also surfaces OOD failures of CUDA-trained LLMs on tasks that are otherwise textbook.

Limitations and future work. The static per-chip ceilings do not account for SLC residency at small sizes (see the LBM 256^2 result above); a workload-aware roofline [4] per task and per size would tighten the score. The single-population $(1+1)$ loop plateaus within 10–25 iterations on most tasks — an island-model or FunSearch-style [1] archive with a novelty signal [2] is the natural next step, particularly for the irregular-memory tasks. Future tasks include sparse linear algebra (SpMV, CG) and cross-chip generalisation (evolve on M2 Pro, evaluate on M3 Max).

References

[1] Romera-Paredes, B. et al. (2024) Mathematical discoveries from program search with large language models. *Nature* **625**, 468–475.

[2] Novikov, A. et al. (2025) AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv:2506.13131*.

[3] Ouyang, A. et al. (2025) KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv:2502.10517*.

[4] Williams, S., Waterman, A. & Patterson, D. (2009) Roofline: An insightful visual performance model for multicore architectures. *CACM* **52**(4), 65–76.

[5] Datta, K. et al. (2008) Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. *Proc. SC '08*.

[6] Nyland, L., Harris, M. & Prins, J. (2007) Fast N-body simulation with CUDA. *GPU Gems 3*, ch. 31.

[7] Schönherr, M. et al. (2011) Multi-thread implementations of the lattice Boltzmann method on non-uniform grids for CPUs and GPUs. *Comput. Math. Appl.* **61**(12), 3730–3743.

[8] Anderson, J. A., Lorenz, C. D. & Travesset, A. (2008) General purpose molecular dynamics simulations fully implemented on graphics processing units. *J. Comput. Phys.* **227**(10), 5342–5359.

[9] Micikevicius, P. (2009) 3D finite difference computation on GPUs using CUDA. *Proc. GPGPU-2*.

[10] Geb, L. et al. (2022) Seamless GPU acceleration for C++ based physics on the Apple M1's unified processing units. *arXiv:2206.01791*.